المذكرة الاحترافية

Software Engineering for Machine Learning Engineers

الفصل الأول: Clean Code

من كتابة كود يعمل، إلى كتابة كود يُبنى عليه نظام إنتاج كامل

Naming • Functions • Classes • Readability • Maintainability

جدول المحتويات

جدول المحتويات 2
مقدمة الفصل الأول: من Notebook إلى Production 5
١،١ Naming — تسمية العناصر البرمجية 6
١. ليه موضوع الـ Naming مهم؟ 6
٢. المشكلة اللي بيحلها Naming الصح 6
٣. إزاي كانت الشركات بتتعامل قبل وجود معايير Naming واضحة 6
٤. ليه الطريقة القديمة مش مناسبة؟ 6
٥. الفكرة الأساسية وراء Clean Naming 6
٦. المفاهيم الأساسية في Naming 6
٧. المكونات: قواعد التسمية حسب نوع العنصر 7
٨. إزاي المبدأ ده بيشتغل داخليًا في عملية القراءة 7
٩. إزاي بيتم استخدامه داخل الشركات الكبرى 7
١٠. فين موقعه داخل مشروع ML حقيقي 7
١١. امتى تستخدم قواعد Naming الصارمة 7
١٢. امتى ممكن تتساهل شوية 7
١٣. أفضل الممارسات 7
١٤. أشهر الأخطاء الشائعة 8
١٥. اعتبارات الأداء 8
١٦. الاعتبارات الأمنية 8
١٧. اعتبارات قابلية التوسع 8
١٨. مقارنة مع نهج بديل 8
١٩. مثال عملي صغير 8
٢٠. مثال من مشروع Production حقيقي 9
٢١. ملخص الموضوع 9
١،٢ Functions — تصميم الدوال 10
١. ليه موضوع تصميم الدوال مهم؟ 10
٢. المشكلة اللي بتحلها الدوال الصغيرة والواضحة 10
٣. إزاي كانت الشركات بتتعامل قبل تبني مبدأ الدوال الصغيرة 10
٤. ليه الطريقة القديمة مش مناسبة للمشاريع الكبيرة 10
٥. الفكرة الأساسية وراء تصميم الدوال الجيدة 10
٦. المفاهيم الأساسية 10
٧. المكونات الأساسية لدالة نظيفة 10
٨. إزاي المبدأ ده بيشتغل داخليًا 11
٩. إزاي بيتم استخدامه داخل الشركات الكبرى 11
١٠. فين موقعه داخل مشروع ML حقيقي 11
١١. امتى تلتزم بتقسيم صارم للدوال 11
١٢. امتى ممكن تكتب دالة أطول شوية 11
١٣. أفضل الممارسات 11
١٤. أشهر الأخطاء الشائعة 11
١٥. اعتبارات الأداء 12

١٦. الاعتبارات الأمنية 12
١٧. اعتبارات قابلية التوسع 12
١٨. مقارنة مع نهج بديل 12
١٩. مثال عملي صغير 12
٢٠. مثال من مشروع Production حقيقي 13
٢١. ملخص الموضوع 13
١,٣ Classes — تصميم الكلاسات 14
١. ليه تصميم الكلاسات مهم؟ 14
٢. المشكلة اللي بيحلها التصميم الجيد للكلاسات 14
٣. إزاي كانت الشركات بتتعامل قبل مبادئ تصميم الكلاسات الحديثة 14
٤. ليه الطريقة القديمة مش مناسبة 14
٥. الفكرة الأساسية وراء تصميم الكلاسات الجيد 14
٦. المفاهيم الأساسية 14
٧. المكونات الأساسية للكلاس 14
٨. إزاي المبدأ ده بيشتغل داخليًا 15
٩. إزاي بيتم استخدامه داخل الشركات الكبرى 15
١٠. فين موقعه داخل مشروع ML حقيقي 15
١١. امتى تستخدم كلاس بدل دالة بسيطة 15
١٢. امتى الدالة البسيطة أفضل من الكلاس 15
١٣. أفضل الممارسات 15
١٤. أشهر الأخطاء الشائعة 15
١٥. اعتبارات الأداء 16
١٦. الاعتبارات الأمنية 16
١٧. اعتبارات قابلية التوسع 16
١٨. مقارنة مع نهج بديل 16
١٩. مثال عملي صغير 16
٢٠. مثال من مشروع Production حقيقي 17
٢١. ملخص الموضوع 17
١,٤ Readability — قابلية القراءة 18
١. ليه قابلية القراءة مهمة؟ 18
٢. المشكلة اللي بتحلها قابلية القراءة العالية 18
٣. إزاي كانت الشركات بتتعامل قبل الاهتمام بالـ Readability كمعيار جودة 18
٤. ليه الطريقة القديمة مش مناسبة 18
٥. الفكرة الأساسية وراء قابلية القراءة العالية 18
٦. المفاهيم الأساسية 18
٧. المكونات الأساسية للكود المقروء 18
٨. إزاي المبدأ ده بيشتغل داخليًا في عملية الفهم 19
٩. إزاي بيتم استخدامه داخل الشركات الكبرى 19
١٠. فين موقعه داخل مشروع ML حقيقي 19
١١. امتى الاهتمام بالـ Readability يكون أولوية قصوى 19
١٢. امتى ممكن تتساهل شوية 19

١٣. أفضل الممارسات 19
١٤. أشهر الأخطاء الشائعة 19
١٥. اعتبارات الأداء 20
١٦. الاعتبارات الأمنية 20
١٧. اعتبارات قابلية التوسع 20
١٨. مقارنة مع نهج بديل 20
١٩. مثال عملي صغير 20
٢٠. مثال من مشروع Production حقيقي 20
٢١. ملخص الموضوع 20
١,٥ Maintainability — قابلية الصيانة 22
١. ليه قابلية الصيانة مهمة؟ 22
٢. المشكلة اللي بتحلها قابلية الصيانة العالية 22
٣. إزاي كانت الشركات بتتعامل قبل الاهتمام بقابلية الصيانة 22
٤. ليه الطريقة القديمة مش مناسبة على المدى الطويل 22
٥. الفكرة الأساسية وراء قابلية الصيانة 22
٦. المفاهيم الأساسية 22
٧. المكونات الأساسية لكود قابل للصيانة 22
٨. إزاي المبدأ ده بيشتغل داخليًا 23
٩. إزاي بيتم استخدامه داخل الشركات الكبرى 23
١٠. فين موقعه داخل مشروع ML حقيقي 23
١١. امتى الاستثمار في قابلية الصيانة يكون أولوية 23
١٢. امتى ممكن تقبل مستوى أقل من الاهتمام بالصيانة 23
١٣. أفضل الممارسات 23
١٤. أشهر الأخطاء الشائعة 23
١٥. اعتبارات الأداء 24
١٦. الاعتبارات الأمنية 24
١٧. اعتبارات قابلية التوسع 24
١٨. مقارنة مع نهج بديل 24
١٩. مثال عملي صغير 24
٢٠. مثال من مشروع Production حقيقي 25
٢١. ملخص الموضوع 25
مشروع عملي على الفصل: إعادة هيكلة Data Preprocessing Script 26
خطوات التنفيذ 26
تحديات شائعة عند التطبيق 26
إزاي الفصل ده بيتستخدم داخل مشروع Machine Learning حقيقي 26
ملخص الفصل الأول وربطه بالفصل القادم 26

مقدمة الفصل الأول: من Notebook إلى Production

الفصل اللي هنبدأ بيه المذكرة اسمه Clean Code، وهو أول حجر في طريق تحويلك من حد بيكتب كود Machine Learning جوه Notebook لحد بيكتب كود يقدر يتشاله فريق كامل جواه شركة تقنية كبيرة.

أنت دلوقتي عندك خلفية قوية في Python وAlgorithms وData Structures وML وDL، وده ممتاز، بس فيه فرق كبير بين إنك تكتب كود بيشتغل (Code that works) وكود قابل للصيانة والتوسع (Maintainable and Scalable Code). الفرق ده هو اللي بيفرق بين Data Scientist بيبني نموذج على جهازه، وML Engineer بيبني نظام هيشتغل جوه Production ويتلمسه عشرات المهندسين على مدار سنين.

ليه بنبدأ بالموضوع ده تحديدًا؟ لأن كل حاجة هنشرحها بعد كده - Clean Code، Design Patterns، SOLID Principles، Testing، Git، Architecture - كلها مبنية فوق أساس واحد: إنك تقدر تكتب كود واضح، منظم، وسهل القراءة. لو الأساس ده مش موجود، أي مبدأ متقدم هتتعلمه هيتحول لمجرد قواعد نظرية من غير تطبيق حقيقي.

اللي هتاخده من الفصل ده هيتبني عليه مباشرة في الفصل الجاي (SOLID Principles)، لأن SOLID في الآخر هو امتداد طبيعي لأفكار الـ Clean Code لكن على مستوى تصميم الكلاسات والعلاقات بينها. وهيتبني عليه كمان لما نوصل لـ Design Patterns وClean Architecture، لأن كل نمط تصميم (Design Pattern) في الآخر هو مجموعة من الكلاسات والدوال، ولو مكتوبة بشكل مش نظيف، النمط نفسه هيبقى صعب الفهم والصيانة.

١,١ Naming — تسمية العناصر البرمجية

Naming هو أبسط موضوع في الفصل شكليًا، لكنه غالبًا أكتر حاجة بتفرق بين كود شركة كبيرة وكود مشروع تخرج. اسم المتغير أو الدالة أو الكلاس هو أول حاجة بيقابلها أي مهندس تاني (أو أنت نفسك بعد ٦ شهور) لما يفتح الملف.

١. ليه موضوع الـ Naming مهم؟

الكود بيتقرأ أضعاف المرات اللي بيتكتب فيها. مهندس واحد بيكتب دالة مرة واحدة، لكن نفس الدالة دي هتتقرأ من عشرات المهندسين على مدار سنين: زميل بيعمل Code Review، مهندس تاني بيصلح Bug، مهندس جديد بينضم للفريق وبيحاول يفهم النظام. لو الأسماء مش واضحة، كل قراءة من دول بتاخد وقت أطول وبتزود احتمال سوء الفهم.

في شركات زي Google وAmazon، فيه مصطلح شائع اسمه Code Review Velocity - يعني السرعة اللي بيتم بيها مراجعة والموافقة على الكود. أسماء غامضة أو مضللة بتبطئ الـ Velocity ده بشكل مباشر، لأن المراجع (Reviewer) بيضطر يسأل ده بيعمل إيه بدل ما يفهم من الاسم نفسه.

٢. المشكلة اللي بيحلها Naming الصح

المشكلة الأساسية هي: الكود بيوصف إزاي (How) بطبيعته، لكنه مش بيوصف ليه (Why) أو إيه (What) إلا لو الأسماء نفسها بتحمل المعنى ده. لو عندك متغير اسمه x أو data أو temp، فالكود بيشتغل بس مفيش حد هيفهم قصدك من غير ما يقرأ كل سطر واحد ويتتبع القيمة.

في مشاريع Machine Learning تحديدًا، المشكلة دي بتتضاعف: عندك Arrays، Tensors، DataFrames، Hyperparameters، كلهم أرقام أو مصفوفات في الآخر. لو اسميتهم df أو arr أو val، مفيش حد هيعرف ده Training data ولا Validation data ولا Predictions من غير ما يرجع يقرأ الكود اللي فوق.

٣. إزاي كانت الشركات بتتعامل قبل وجود معايير Naming واضحة

في مراحل مبكرة من صناعة البرمجيات، وكمان دلوقتي في السكربتات السريعة اللي بيكتبها الـ Data Scientists، كان الاعتماد بيبقى على أسماء مختصرة جدًا (زي i, j, tmp, df1, df2) لأن الهدف كان السرعة في الكتابة مش وضوح القراءة لاحقًا. الفرضية الضمنية كانت إن الكود ده هيتنفذ مرة واحدة وخلاص.

المشكلة إن السكربتات دي غالبًا بتتحول لأنظمة حقيقية بمرور الوقت (يُعرف الأمر ده أحيانًا بـ Technical Debt الناتج عن الانتقال من Prototype لـ Production)، وساعتها بيكتشف الفريق إن محدش قادر يفهم إيه اللي df3 بيمثله بعد ٣ شهور.

٤. ليه الطريقة القديمة مش مناسبة؟

الاختصار المفرط أو التسمية العشوائية بتنقل العبء الذهني (Cognitive Load) من وقت الكتابة لوقت القراءة، وبما إن القراءة بتحصل عدد مرات أكبر بكتير من الكتابة، فالنتيجة الصافية إنك بتوفر دقايق وقت الكتابة مقابل ساعات ضايعة وقت القراءة والصيانة لاحقًا - وده بالظبط عكس الهدف من الهندسة البرمجية الاحترافية.

٥. الفكرة الأساسية وراء Clean Naming

الفكرة الأساسية بسيطة: الاسم لازم يجاوب على سؤال إيه ده وليه موجود من غير ما تحتاج تقرا الكود اللي جواه. يعني بدل df استخدم raw_training_data، بدل temp استخدم normalized_pixel_values، بدل calc استخدم calculate_validation_accuracy.

المبدأ ده معروف باسم Self-Documenting Code، وهو الكود اللي بيشرح نفسه من غير ما يحتاج كومنتات كتيرة توضح الأسماء الغامضة.

٦. المفاهيم الأساسية في Naming

- Intention-Revealing Names: الاسم لازم يوضح النية من وجود المتغير أو الدالة، مش بس نوعه.

- Pronounceable Names: اسم ينطق بسهولة في نقاش الفريق، مثل generation_timestamp بدل genymdhms.
- Searchable Names: تجنب أسماء قصيرة جدًا زي e، صعب تعمل عليها بحث في مشروع كبير من غير نتائج غلط كتير.
- Consistent Vocabulary: استخدام نفس الكلمة لنفس المفهوم في كل المشروع، ومتستخدمش fetch في مكان وretrieve في مكان تاني لنفس المعنى.
- No Mental Mapping: القارئ ميحتاجش يعمل ترجمة ذهنية زي إنه يفتكر إن x هي عدد الـ epochs.

٧. المكونات: قواعد التسمية حسب نوع العنصر

- Variables وAttributes: اسم واضح كصفة أو اسم، مثل model_accuracy، learning_rate.
- Functions وMethods: تبدأ بفعل يوضح الفعل اللي بتعمله، مثل train_model، load_dataset، normalize_features.
- Classes: اسم يمثل الكيان أو المفهوم، مثل ModelTrainer، DataPreprocessor، بدون فعل.
- Booleans: تبدأ بـ is أو has أو can لتوضيح إنها قيمة منطقية، مثل has_missing_values، is_trained.
- Constants: أحرف كابيتال مفصولة بـ underscore، مثل MAX_BATCH_SIZE، DEFAULT_LEARNING_RATE.

٨. إزاي المبدأ ده بيشتغل داخليًا في عملية القراءة

لما مهندس بيفتح ملف كود، دماغه بتعمل عملية اسمها Pattern Matching، بتحاول تربط الاسم بمفهوم معروف عنده مسبقًا. اسم زي calculate_cross_entropy_loss بيوفر المعنى فورًا لأي حد عنده خلفية ML، من غير ما يحتاج يفتح جسم الدالة. الاسم هنا بيشتغل كطبقة تجريد (Abstraction) تخلي القارئ يفهم إيه من غير ما يهتم بإزاي إلا لو احتاج فعلاً.

٩. إزاي بيتم استخدامه داخل الشركات الكبرى

شركات زي Google وMeta عندها Style Guides رسمية ومنشورة جزئيًا بتحدد قواعد صارمة للـ Naming، وده بيتفحص أوتوماتيكيًا عن طريق أدوات Linting زي pylint أو flake8 بتشتغل كجزء من الـ CI Pipeline قبل ما أي Pull Request يتم قبوله. الأسماء مش مجرد تفضيل شخصي، دي جزء من معايير الجودة اللي بيتفحصها النظام أوتوماتيكيًا.

١٠. فين موقعه داخل مشروع ML حقيقي

بيظهر في كل حتة تقريبًا: أسماء الـ Feature columns في الـ Data Pipeline، أسماء الـ Functions في وحدات الـ Preprocessing وTraining وEvaluation، أسماء الـ Model artifacts وMLflow experiment runs، وحتى أسماء الـ environment variables في ملفات الـ Configuration.

١١. امتى تستخدم قواعد Naming الصارمة

- في أي كود هيدخل الـ Repository الرسمي للمشروع أو هيتشارك مع فريق.
- في أي كود Production هيشتغل لفترة طويلة ويحتاج صيانة.
- في أي مكتبة أو API هيستخدمها مهندسين تانيين.

١٢. امتى ممكن تتساهل شوية

- في Exploratory Data Analysis سريع جوه Notebook شخصي، مش هيتشارك ولا هيدخل الـ Production، ساعتها اختصارات زي df أو X_train مقبولة لأنها اصطلاحية ومفهومة ضمنيًا في مجتمع الـ Data Science.
- لكن حتى هنا، لو الـ Notebook هيتحول لكود Production لاحقًا، لازم يتعاد تسمية العناصر قبل الدمج.

١٣. أفضل الممارسات

Best Practices

- استخدم أسماء طويلة وواضحة بدل أسماء قصيرة وغامضة، الطول مش عيب لو الوضوح زاد.
- خلي اسم الدالة يعكس بالظبط اللي بتعمله، ولو الاسم بقى طويل جدًا عشان يوصف كذا حاجة، ده مؤشر إن الدالة نفسها لازم تتقسم.
- التزم بمصطلح واحد لكل مفهوم في المشروع كله.
- استخدم Linter مثل pylint للتأكد من الالتزام بمعايير التسمية أوتوماتيكيًا.

١٤. أشهر الأخطاء الشائعة

Common Mistakes

- استخدام أسماء عامة جدًا زي data, temp, val, result في أكتر من مكان بمعاني مختلفة.
- استخدام اختصارات شخصية مفهومة بس للكاتب نفسه زي clc_ls بدل calculate_loss.
- تسمية Boolean variable باسم عادي بدل ما يوضح إنها قيمة منطقية زي trained بدل is_trained.
- تغيير الغرض من المتغير في نص الكود من غير ما يتغير اسمه.

١٥. اعتبارات الأداء

أسماء المتغيرات نفسها مالهاش أي تأثير على أداء البرنامج وقت التشغيل، لأن أسماء المتغيرات بتتحول لعناوين ذاكرة في الآخر بغض النظر عن طولها. لكن الأداء غير المباشر بيتأثر: كود بأسماء واضحة بيتصلح فيه الأخطاء بسرعة، يعني وقت أقل ضايع في Debugging وتشخيص المشاكل.

١٦. الاعتبارات الأمنية

التأثير الأمني غير مباشر لكنه موجود: أسماء متغيرات واضحة بتقلل احتمال إن مهندس يسيء فهم قيمة حساسة زي raw_user_data غير المعالجة ويستخدمها في مكان غلط. في المقابل، لازم الانتباه لعدم كتابة معلومات حساسة داخل الأسماء نفسها أو في الكومنتات، زي أسماء متغيرات بتحتوي على مفاتيح API أو كلمات مرور بشكل حرفي.

١٧. اعتبارات قابلية التوسع

لما المشروع بيكبر ويدخل فيه مهندسين جدد، معايير التسمية المتسقة بتخلي عملية الـ Onboarding أسرع بكتير، لأن أي مهندس جديد بيقدر يتوقع اسم الدالة أو المتغير من غير ما يسأل زمايله باستمرار. من غير الاتساق ده، كل ما الفريق يكبر، سرعة الفهم بتقل بشكل غير متناسب مع حجم الفريق.

١٨. مقارنة مع نهج بديل

البديل الوحيد الحقيقي هو الاعتماد على الكومنتات الكتيرة لشرح أسماء غامضة بدل تسميتها صح من الأول. المشكلة إن الكومنتات بتتقادم بمرور الوقت لأن محدش بيحدثها لما الكود يتغير، لكن اسم واضح جزء لا يتجزأ من الكود نفسه فمستحيل يتقادم أو يتناسى تحديثه. لذلك الشركات الكبرى بتفضل دايمًا الأسماء الموثقة ذاتيًا على الاعتماد المفرط على الكومنتات.

١٩. مثال عملي صغير

```
# Before - vague names
def calc(d, t):
    r = d[d['col'] > t]
    return r
```

```
# After - clear, intention-revealing names
def filter_transactions_above_threshold(transactions_df, amount_threshold):
    filtered_transactions = transactions_df[
        transactions_df['amount'] > amount_threshold
    ]
    return filtered_transactions
```

٢٠. مثال من مشروع Production حقيقي

في Pipeline حقيقي لتدريب نموذج Recommendation System، هتلاقي أسماء زي user_interaction_features، negative_sampling_ratio، candidate_item_embeddings بدل u_feat أو c_emb أو neg_ratio. الأسماء دي مش رفاهية، دي اللي بتخلي مهندس جديد ينضم للفريق ويقدر يفهم الـ Data Pipeline في يومه الأول بدل ما ياخد أسابيع في اجتماعات شرح.

٢١. ملخص الموضوع

خلاصة Naming

- الاسم لازم يجاوب سؤال إيه وليه من غير ما تقرأ الكود اللي جواه.
- الاتساق في المصطلحات عبر المشروع أهم من أي قاعدة فردية.
- الطول مش مشكلة، الغموض هو المشكلة.
- أدوات الـ Linting بتحول القاعدة دي من تفضيل شخصي لمعيار جودة مفروض أوتوماتيكيًا.

١,٢ Functions — تصميم الدوال

الدالة (Function) هي وحدة البناء الأساسية لأي برنامج. لو الأسماء هي الكلمات، الدوال هي الجمل اللي بتحكي قصة الكود. دالة مكتوبة صح بتقرأ زي فقرة واضحة، ودالة مكتوبة غلط بتتحول لغز لازم تفكه كل مرة تفتحها.

١. ليه موضوع تصميم الدوال مهم؟

الدالة هي أصغر وحدة قابلة لإعادة الاستخدام (Reusability) والاختبار (Testability) في الكود. لو الدالة مصممة صح، تقدر تختبرها لوحدها، تفهمها لوحدها، وتستبدلها لوحدها من غير ما تكسر باقي النظام. ده أساس أي نظام Machine Learning حقيقي، لأن الـ Pipeline بيتكون من عشرات الخطوات (تحميل بيانات، تنظيف، تدريب، تقييم) وكل خطوة لازم تكون دالة أو مجموعة دوال واضحة الحدود.

في بيئة الشركات، الدوال الصغيرة والواضحة هي اللي بتخلي فريق كامل يقدر يشتغل على نفس الـ Codebase في نفس الوقت من غير ما يتصادموا مع بعض، لأن كل مهندس بيقدر يعدل دالة معينة وهو واثق إنها معزولة عن باقي النظام.

٢. المشكلة اللي بتحلها الدوال الصغيرة والواضحة

المشكلة هي الدوال العملاقة (Long Functions أو God Functions) اللي بتعمل كذا حاجة في نفس الوقت: بتحمل بيانات، وتنضفها، وتدرب النموذج، وتحسب المقاييس، كل ده في دالة واحدة اسمها run_everything. الدالة دي بتبقى صعبة الفهم، صعبة الاختبار، وأي تعديل بسيط فيها ممكن يكسر جزء تاني ماكانش متوقع إنه هيتأثر.

٣. إزاي كانت الشركات بتتعامل قبل تبني مبدأ الدوال الصغيرة

في السكربتات التقليدية وبرمجيات الأنظمة القديمة (Legacy Systems)، كان شائع كتابة سكربتات طويلة تنفذ بالتتابع من أول السطر للآخر من غير تقسيم حقيقي لدوال منفصلة، وده كان مقبول وقتها لأن البرامج كانت أبسط وأصغر وبيشتغل عليها مهندس واحد أو اتنين بس.

٤. ليه الطريقة القديمة مش مناسبة للمشاريع الكبيرة

مع زيادة حجم المشروع وعدد المهندسين، السكربت الطويل بيتحول لعنق زجاجة: أي تعديل بيحتاج فهم كامل السكربت من الأول للآخر، والاختبار بيبقى شبه مستحيل لأنك مش قادر تختبر جزء لوحده. في مشاريع ML، ده بيبقى خطير جدًا لأن خطأ بسيط في مرحلة التنظيف ممكن يفسد نتيجة التدريب كله من غير ما حد يلاحظ بسرعة.

٥. الفكرة الأساسية وراء تصميم الدوال الجيدة

المبدأ الأشهر هنا اسمه Single Responsibility لكل دالة: كل دالة لازم تعمل حاجة واحدة بس، وتعملها كويس. لو حسيت إنك بتشرح الدالة باستخدام كلمة و (and)، زي دي الدالة بتحمل البيانات وتنضفها، ده مؤشر إنها لازم تتقسم لدالتين منفصلتين.

٦. المفاهيم الأساسية

- Single Level of Abstraction: كل سطر جوه الدالة لازم يكون في نفس مستوى التفصيل، متخلطش تفاصيل منخفضة المستوى (زي فتح ملف) مع منطق عالي المستوى (زي حساب دقة النموذج) في نفس الدالة.
- Small Function Size: الدالة المثالية بتترى في الشاشة من غير Scroll، وغالبًا أقل من ٢٠ سطر.
- Few Parameters: قلة عدد المدخلات (Parameters) بتقلل التعقيد، ولو الدالة محتاجة أكتر من ٣-٤ مدخلات، فكر في تجميعهم في كائن (Object) واحد.
- No Side Effects: الدالة المثالية ميكونش ليها تأثيرات جانبية غير متوقعة، يعني متغيرش حاجة برا نطاقها من غير ما يكون ده هو الغرض المعلن منها.

٧. المكونات الأساسية لدالة نظيفة

- التوقيع (Signature): اسم الدالة والمدخلات ونوع المخرجات (Type Hints في Python).
- الجسم (Body): المنطق الفعلي، لازم يكون بسيط ومباشر.

- القيمة المرجعة (Return Value): واضحة ومتسقة، ميرجعش نوع بيانات مختلف حسب الحالة.
- التوثيق (Docstring): شرح مختصر لوظيفة الدالة، خصوصًا لو هتستخدم من فريق تاني.

٨. إزاي المبدأ ده بيشتغل داخليًا

لما تقسم منطق معقد لدوال صغيرة، إنت في الحقيقة بتبني هرم من التجريد (Abstraction Hierarchy): الدالة الرئيسية بتنادي على دوال فرعية بأسماء واضحة، وكل دالة فرعية ممكن تنادي على دوال أصغر. القارئ يقدر يفهم الصورة الكاملة من قراءة الدالة الرئيسية بس، وبعدين يغوص جوه أي دالة فرعية لو احتاج التفاصيل.

٩. إزاي بيتم استخدامه داخل الشركات الكبرى

في فرق الـ ML Engineering داخل شركات زي Uber أو Netflix، الـ Training Pipeline بيتقسم لدوال ومراحل واضحة (Data Ingestion، Feature Engineering، Model Training، Evaluation، Deployment)، وكل مرحلة غالبًا بتتحول لـ Task منفصلة داخل أدوات تنسيق سير العمل (Workflow Orchestration) زي Airflow أو Kubeflow Pipelines. التقسيم ده مش بس تنظيم كود، دي بنية أساسية تسمح بإعادة تشغيل مرحلة واحدة لوحدها لو فشلت من غير إعادة تشغيل الـ Pipeline كله.

١٠. فين موقعه داخل مشروع ML حقيقي

موجود في كل طبقة: دوال تحميل البيانات، دوال المعالجة المسبقة، دوال بناء النموذج، دوال التدريب، دوال التقييم وحساب المقاييس، ودوال حفظ ونشر النموذج. كل دالة من دول لازم تكون قابلة للاختبار بمعزل عن الباقي.

١١. امتى تلتزم بتقسيم صارم للدوال

- لما تكتب كود هيدخل Production أو هيتشارك مع فريق.
- لما المنطق معقد بما يكفي إنه يحتاج اختبار منفصل لكل جزء.
- لما تتوقع إن جزء من المنطق ممكن يتغير أو يتطور مستقبلًا.

١٢. امتى ممكن تكتب دالة أطول شوية

- في تحليل استكشافي سريع (Quick EDA) جوه Notebook شخصي مش هيتشارك.
- لما التقسيم المفرط هيزود التعقيد بدل ما يقلله، زي تقسيم منطق بسيط جدًا لثلاث دوال منفصلة من غير داعي حقيقي.

١٣. أفضل الممارسات

Best Practices

- خلي كل دالة تعمل حاجة واحدة بس، ولو محتاجة كلمة و لشرحها، قسمها.
- استخدم Type Hints في Python لتوضيح نوع المدخلات والمخرجات.
- قلل عدد المدخلات، ولو زادوا عن ٣-٤، جمعهم في كائن أو Data Class.
- اكتب Docstring مختصر يوضح الغرض من الدالة، خصوصًا للدوال العامة (Public Functions).

١٤. أشهر الأخطاء الشائعة

Common Mistakes

- كتابة دوال طويلة تعمل كذا حاجة في نفس الوقت (God Functions).
- استخدام Flags أو Boolean parameters بتغير سلوك الدالة بالكامل بدل عمل دالتين منفصلتين.
- تعديل متغيرات خارجية (Global State) من جوه الدالة من غير توضيح واضح.

- تكرار نفس المنطق في أكتر من دالة بدل استخراجه في دالة مشتركة (خرق مبدأ DRY - Don't Repeat Yourself).

١٥. اعتبارات الأداء

تقسيم الكود لدوال صغيرة بيضيف تكلفة بسيطة جدًا في استدعاء الدوال (Function Call Overhead)، لكن التكلفة دي غالبًا مهملة مقارنة بفايدة الوضوح وسهولة الصيانة. في حالات نادرة جدًا في كود حساس للأداء بشكل قصوى (زي حلقات معالجة بيانات ضخمة جدًا)، ممكن الدمج المدروس يكون له مبرر، لكن ده استثناء مش قاعدة.

١٦. الاعتبارات الأمنية

الدوال الصغيرة والمحددة بوضوح بتسهل تحديد نقاط الدخول (Entry Points) اللي محتاجة التحقق من صحة المدخلات (Input Validation)، خصوصًا الدوال اللي بتستقبل بيانات من مصادر خارجية زي طلبات المستخدم أو ملفات مرفوعة. الدالة العملاقة اللي بتخلط استقبال البيانات مع معالجتها بتخلي صعب تحديد فين بالظبط لازم تتحقق من صحة المدخلات.

١٧. اعتبارات قابلية التوسع

الدوال الصغيرة قابلة لإعادة الاستخدام في أماكن مختلفة من المشروع، وده بيقلل التكرار وبيسهل التوسع لما تحتاج تضيف ميزة جديدة تعتمد على منطق موجود بالفعل. النظام المبني من دوال واضحة الحدود أسهل بكتير في التوزيع على فرق متعددة، كل فريق مسؤول عن مجموعة دوال أو وحدات معينة.

١٨. مقارنة مع نهج بديل

البديل هو السكربت الواحد المتسلسل (Monolithic Script) اللي بينفذ كل حاجة بالترتيب من غير تقسيم لدوال. الأسلوب ده أسرع في الكتابة الأولى لمشروع صغير جدًا أو تجربة سريعة، لكنه بيفشل بسرعة مع زيادة حجم المشروع لأن الاختبار والصيانة بيبقوا شبه مستحيلين. الشركات الكبرى دايمًا بتختار التقسيم الواضح رغم التكلفة الإضافية البسيطة وقت الكتابة، لأن الفايدة على المدى الطويل أكبر بكتير.

١٩. مثال عملي صغير

```
# Before - one function doing too many things
def process(path):
    df = pd.read_csv(path)
    df = df.dropna()
    df['scaled'] = (df['x'] - df['x'].mean()) / df['x'].std()
    model = LinearRegression().fit(df[['scaled']], df['y'])
    return model

# After - split into single-responsibility functions
def load_dataset(path: str) -> pd.DataFrame:
    return pd.read_csv(path)

def remove_missing_values(df: pd.DataFrame) -> pd.DataFrame:
    return df.dropna()

def standardize_column(df: pd.DataFrame, column: str) -> pd.DataFrame:
    df[f'{column}_scaled'] = (df[column] - df[column].mean()) / df[column].std()
    return df

def train_linear_model(df: pd.DataFrame, feature: str, target: str):
    return LinearRegression().fit(df[[feature]], df[target])
```

٢٠. مثال من مشروع Production حقيقي

في نظام Fraud Detection حقيقي، الـ Feature Engineering Pipeline بيتقسم لعشرات الدوال الصغيرة: دالة لحساب متوسط قيمة المعاملات آخر ٣٠ يوم، دالة لحساب عدد المعاملات في آخر ساعة، دالة لحساب المسافة الجغرافية بين آخر معاملتين. كل دالة قابلة للاختبار لوحدها، وده اللي بيخلي فريق كامل يقدر يضيف Feature جديدة من غير ما يخاف يكسر Features التانية.

٢١. ملخص الموضوع

خلاصة Functions

- كل دالة لازم تعمل حاجة واحدة بس وتعملها كويس.
- قلة المدخلات ووضوح المخرجات بيسهلوا الفهم والاختبار.
- الدوال الصغيرة قابلة لإعادة الاستخدام وأسهل في التوزيع بين الفرق.
- الدالة العملاقة اللي بتعمل كذا حاجة هي علامة تحذير تحتاج إعادة هيكلة (Refactoring) فورية.

١,٣ Classes — تصميم الكلاسات

لو الدوال هي الجمل، الكلاس (Class) هو الفقرة اللي بتجمع جمل مرتبطة ببعض حوالين فكرة واحدة. الكلاس مش مجرد صندوق يجمع دوال ومتغيرات، هو تمثيل لمفهوم حقيقي في المشروع، وتصميمه صح بيحدد مدى سهولة توسيع النظام كله لاحقًا.

١. ليه تصميم الكلاسات مهم؟

الكلاس هو الوحدة اللي بتجمع البيانات (State) مع السلوك (Behavior) اللي بيتعامل مع البيانات دي في مكان واحد. في مشروع ML، عندك مفاهيم حقيقية زي Dataset وModel وTrainer وEvaluator، وكل واحد منهم أفضل تمثيل ليه هو كلاس بيحمل الحالة الخاصة بيه (زي مسار البيانات أو أوزان النموذج) ومعاه الدوال اللي بتتعامل مع الحالة دي.

كلاس مصمم صح بيوفر حدود واضحة (Boundaries) بين أجزاء النظام المختلفة، وده اللي بيسمح لفرق مختلفة تشتغل على أجزاء مختلفة من نفس المشروع من غير ما تتداخل مسؤولياتهم.

٢. المشكلة اللي بيحلها التصميم الجيد للكلاسات

المشكلة هي الكلاسات العملاقة (God Classes أو God Objects) اللي بتحاول تعمل كل حاجة: كلاس واحد اسمه MLPipeline بيحمل البيانات، وينضفها، ويدرب النموذج، وينشره، ويولد التقارير. أي تعديل بسيط في جزء واحد بيحتاج فهم الكلاس كله، وأي Bug في جزء بيأثر على أجزاء تانية من غير ما يكون متوقع.

٣. إزاي كانت الشركات بتتعامل قبل مبادئ تصميم الكلاسات الحديثة

في البرمجة الإجرائية التقليدية (Procedural Programming) والمشاريع المبكرة، كان الاعتماد على دوال منفصلة تتعامل مع بيانات مشتركة (Global State) من غير تجميع واضح. مع تبني البرمجة الكائنية (Object-Oriented Programming)، بدأ الميل لتجميع كل حاجة في كلاس واحد كبير كحل سريع من غير تفكير عميق في توزيع المسؤوليات.

٤. ليه الطريقة القديمة مش مناسبة

الكلاس العملاق بيخالف مبدأ Single Responsibility من أساسه، وبيخلي إعادة الاستخدام شبه مستحيلة، لأنك مش قادر تستخدم جزء واحد بس من الكلاس (زي التدريب) من غير باقي الأجزاء المرتبطة بيه بشكل غير ضروري. ده كمان بيصعب الاختبار، لأن اختبار جزء واحد بيتطلب تجهيز الكلاس كله بكل حالاته.

٥. الفكرة الأساسية وراء تصميم الكلاسات الجيد

كل كلاس لازم يمثل مفهوم واحد واضح، وليه سبب واحد بس يخليه يتغير (Single Reason to Change - وده أساس مبدأ Single Responsibility اللي هنشرحه بالتفصيل في الفصل الجاي عن SOLID). لو الكلاس بيمثل أكتر من مفهوم، ده علامة إنه محتاج ينقسم لكلاسات أصغر متخصصة.

٦. المفاهيم الأساسية

- Encapsulation: إخفاء التفاصيل الداخلية للكلاس وعرض واجهة (Interface) واضحة بس للتعامل معاه.
- Cohesion: مدى ارتباط الدوال والمتغيرات جوه الكلاس ببعض؛ الكلاس عالي الترابط (High Cohesion) كل عناصره بتخدم نفس الغرض.
- Coupling: مدى اعتماد كلاس على كلاسات تانية؛ الهدف دايمًا تقليل الاعتماد المتبادل (Low Coupling) عشان تقدر تغير كلاس من غير ما تكسر كلاسات تانية.
- Composition over Inheritance: تفضيل بناء الكلاس من مكونات أصغر (Composition) بدل الاعتماد المفرط على الوراثة (Inheritance) اللي بتخلق علاقات جامدة صعب تغييرها.

٧. المكونات الأساسية للكلاس

- Attributes (الحالة): البيانات اللي الكلاس بيحملها، زي model_weights أو dataset_path.

- Methods (السلوك): الدوال اللي بتتعامل مع الحالة دي، زي train() أو predict().
- Constructor (__init__ في Python): بيحدد إزاي الكلاس بيتبني ويتهيأ في البداية.

- Public Interface: الدوال والمتغيرات المفروض تتستخدم من برا الكلاس، مقابل التفاصيل الداخلية الخاصة (Private) اللي مفروض متتلمسش من برا.

٨. إزاي المبدأ ده بيشتغل داخليًا

لما تصمم كلاس بحدود واضحة، إنت بتبني عقد (Contract) ضمني بين الكلاس ده وباقي أجزاء النظام: أي حد بيستخدم الكلاس بيعرف بالظبط إيه اللي يقدر يطلبه منه (عن طريق الـ Public Methods) من غير ما يحتاج يعرف إزاي الكلاس بيعمل ده جوا. ده بيسمح بتغيير التفاصيل الداخلية للكلاس (زي تغيير خوارزمية التدريب) من غير ما يكسر أي كود تاني بيستخدم الكلاس، طالما الـ Public Interface فضل ثابت.

٩. إزاي بيتم استخدامه داخل الشركات الكبرى

في فرق الـ ML Platform داخل شركات زي Meta أو Netflix، بيتبنوا كلاسات معيارية (Standardized Classes) زي BaseModel أو BaseTrainer بتوفر واجهة موحدة، وكل فريق بيبني الكلاس الخاص بمشروعه اعتمادًا على الأساس ده (Inheritance أو Composition)، وده بيسمح ببناء أدوات مشتركة (زي أدوات المراقبة والـ Logging) تشتغل مع أي نموذج طالما ملتزم بالواجهة الموحدة.

١٠. فين موقعه داخل مشروع ML حقيقي

بيظهر في كل مكون رئيسي: كلاس Dataset بيدير تحميل ومعالجة البيانات، كلاس Model بيغلف بنية النموذج، كلاس Trainer بيدير حلقة التدريب (Training Loop)، كلاس Evaluator بيحسب المقاييس، وكلاس Config بيدير الإعدادات.

١١. امتى تستخدم كلاس بدل دالة بسيطة

- لما عندك حالة (State) لازم تتحفظ وتتحدث عبر عدة عمليات، زي عداد أو نموذج مدرب.
- لما عندك سلوك مرتبط ببعض هيتكرر استخدامه بأشكال مختلفة (عن طريق الوراثة أو الواجهات).
- لما تحتاج تمثل مفهوم حقيقي واضح في المشروع بشكل منظم.

١٢. امتى الدالة البسيطة أفضل من الكلاس

- لما العملية بسيطة جدًا ومفيهاش حالة محتاجة تتحفظ (زي دالة تحويل وحدات القياس).
- لما إنشاء كلاس هيزود التعقيد من غير فايدة حقيقية (Over-engineering).

١٣. أفضل الممارسات

Best Practices

- خلي كل كلاس يمثل مفهوم واحد واضح وله سبب واحد بس يخليه يتغير.
- قلل الاعتماد المتبادل بين الكلاسات (Low Coupling) عن طريق التعامل من خلال واجهات واضحة.
- فضل التركيب (Composition) على الوراثة (Inheritance) إلا لو فيه علاقة is-a حقيقية وواضحة.
- خلي الـ Public Interface صغير وواضح، وأخفي التفاصيل الداخلية (Encapsulation).

١٤. أشهر الأخطاء الشائعة

Common Mistakes

- بناء كلاس عملاق (God Class) بيحاول يعمل كل حاجة في المشروع.
- استخدام الوراثة (Inheritance) في حالات مفيهاش علاقة is-a حقيقية، بس عشان توفير كتابة كود.
- كشف التفاصيل الداخلية للكلاس (زي المتغيرات الخاصة) للاستخدام المباشر من برا بدل توفير Methods واضحة.
- خلق اعتماد دائري (Circular Dependency) بين كلاسات بتحتاج بعض في نفس الوقت.

١٥. اعتبارات الأداء

التصميم الكائني الجيد نادرًا ما يكون له تأثير سلبي ملموس على الأداء في لغات زي Python، لأن التكلفة الإضافية لإنشاء الكائنات (Object Instantiation) صغيرة جدًا مقارنة بالعمليات الحسابية الفعلية زي التدريب. في حالات نادرة جدًا حيث الأداء حرج للغاية (زي معالجة بيانات ضخمة في حلقات مكثفة)، ممكن الحاجة لتبسيط البنية، لكن ده استثناء يتقرر بناءً على قياس فعلي (Profiling) مش افتراض مسبق.

١٦. الاعتبارات الأمنية

الـ Encapsulation الجيد بيقلل احتمالية التعامل الخاطئ مع بيانات حساسة، لأن الوصول للبيانات بيمر من خلال Methods محددة ممكن تتحقق من صحة المدخلات وتمنع الوصول غير المصرح بيه. الكلاس اللي بيكشف كل بياناته الداخلية مباشرة بيصعب فرض قواعد تحقق موحدة عليها.

١٧. اعتبارات قابلية التوسع

الكلاسات المصممة بحدود واضحة وترابط منخفض (Low Coupling) بتسمح بتوسيع النظام عن طريق إضافة كلاسات جديدة تلتزم بنفس الواجهة، من غير الحاجة لتعديل الكود الموجود. المبدأ ده هيتوسع بالتفصيل في الفصل الجاي تحت اسم Open/Closed Principle.

١٨. مقارنة مع نهج بديل

البديل هو الاعتماد الكامل على البرمجة الإجرائية (Procedural Programming) بدوال ومتغيرات عامة (Global Variables) من غير تجميع في كلاسات. الأسلوب ده مناسب لسكربتات صغيرة جدًا، لكنه بيفشل في تنظيم الحالة (State Management) في أنظمة معقدة زي Training Pipeline بيحتاج يتتبع حالة النموذج والبيانات والإعدادات في نفس الوقت. الشركات الكبرى بتعتمد على البرمجة الكائنية لأنها بتوفر تنظيم طبيعي للحالة المعقدة.

١٩. مثال عملي صغير

```
# Before - scattered functions and global state
model = None
def train(data):
    global model
    model = LinearRegression().fit(data['X'], data['y'])

# After - a class grouping state and behavior together
class RegressionModelTrainer:
    def __init__(self, learning_rate: float = 0.01):
        self.learning_rate = learning_rate
        self.model = None

    def train(self, features, targets):
        self.model = LinearRegression().fit(features, targets)
        return self.model

    def predict(self, features):
```

```
        if self.model is None:
            raise RuntimeError('Model has not been trained yet')
        return self.model.predict(features)
```

٢٠. مثال من مشروع Production حقيقي

في نظام حقيقي لتوصية المحتوى (Recommendation System)، غالبًا هتلاقي كلاس اسمه CandidateGenerator مسؤول بس عن توليد عناصر مرشحة، وكلاس منفصل اسمه Ranker مسؤول عن ترتيب المرشحين دول، وكلاس تالت اسمه FeatureStore مسؤول عن جلب الخصائص (Features). كل كلاس معزول وقابل للاختبار والاستبدال لوحده، وده اللي بيسمح لفريق يحسن خوارزمية الترتيب من غير ما يلمس منطق توليد المرشحين خالص.

٢١. ملخص الموضوع

خلاصة Classes

- كل كلاس لازم يمثل مفهوم واحد واضح وله سبب واحد بس يخليه يتغير.
- الترابط العالي جوه الكلاس (High Cohesion) والاعتماد المنخفض بين الكلاسات (Low Coupling) هما هدف التصميم الجيد.
- فضل التركيب على الوراثة إلا لو فيه علاقة is-a حقيقية.
- الكلاس العملاق (God Class) هو أخطر علامة تحتاج إعادة هيكلة فورية.

١,٤ Readability — قابلية القراءة

Readability مش موضوع منفصل عن Naming وFunctions وClasses، هو النتيجة الطبيعية لتطبيقهم صح. لكنه كمان بيضيف طبقة إضافية: تنظيم بصري وهيكلي للكود يخلي العين تتبع المنطق من غير مجهود.

١. ليه قابلية القراءة مهمة؟

أي مهندس بيقضي وقت أطول بكتير في قراءة كود موجود عن كتابة كود جديد من الصفر. تقدير شائع في صناعة البرمجيات إن نسبة الوقت المستهلك في القراءة مقابل الكتابة بتكون أكبر بكتير من ١ لـ ١، أحيانًا بتوصل لـ ١٠ لـ ١. يعني استثمار وقت في خلي الكود مقروء بيرجعلك أضعافه في الوقت المستهلك لاحقًا.

في فرق الـ ML تحديدًا، الكود بيتغير باستمرار (تجارب جديدة، Features جديدة، تعديلات على النموذج)، فقابلية القراءة العالية بتخلي كل تعديل أسرع وأقل عرضة للأخطاء.

٢. المشكلة اللي بتحلها قابلية القراءة العالية

الكود اللي بيشتغل بس صعب القراءة (زي أسطر طويلة جدًا، تعشيش عميق من الشروط Nested Conditionals، أو تنسيق غير متسق) بيخلي أي تعديل بسيط يستهلك وقت أطول بكتير في فهم السياق قبل حتى البدء في التعديل نفسه. ده بيبطئ سرعة الفريق ككل، مش بس المهندس اللي كتب الكود.

٣. إزاي كانت الشركات بتتعامل قبل الاهتمام بالـ Readability كمعيار جودة

قبل تبني معايير التنسيق الموحدة (Style Guides) وأدوات الفحص الأوتوماتيكي، كان كل مهندس بيكتب بأسلوبه الشخصي: مسافات بادئة (Indentation) مختلفة، ترتيب استيراد (Imports) عشوائي، تعليقات غير منتظمة. النتيجة كانت كود بأساليب متضاربة داخل نفس المشروع، وكل انتقال بين ملفات مختلفة كان بيحتاج إعادة تكيف ذهني.

٤. ليه الطريقة القديمة مش مناسبة

التنوع في الأسلوب الشخصي من غير معيار موحد بيزود العبء الذهني على أي حد بيقرأ كود مكتوب من زميل مختلف، وبيخلي عملية الـ Code Review أبطأ لأن جزء كبير من الوقت بيروح في ملاحظات على التنسيق بدل التركيز على المنطق الفعلي للكود.

٥. الفكرة الأساسية وراء قابلية القراءة العالية

الكود المقروء بيقلل المسافة بين لحظة ما تشوف الكود ولحظة ما تفهم قصده. ده بيتحقق من خلال تنسيق متسق، تقسيم منطقي واضح (فقرات كود، مسافات فاصلة بين الأجزاء المنطقية المختلفة)، وتجنب التعقيد غير الضروري (زي التعشيش العميق أو الأسطر الطويلة جدًا).

٦. المفاهيم الأساسية

- Consistent Formatting: تنسيق موحد للمسافات والفواصل الأسطر عبر المشروع كله، بيتفرض غالبًا عن طريق أدوات زي black أو prettier.
- Vertical Density: الأسطر المرتبطة منطقيًا ببعض تكون قريبة من بعض، والأجزاء المنفصلة منطقيًا بينهم مسافة فاصلة.
- Avoiding Deep Nesting: تجنب التعشيش العميق للشروط والحلقات (أكتر من ٢-٣ مستويات) لأنه بيصعب تتبع المنطق.
- Early Returns: الخروج من الدالة بدري لما شرط معين يتحقق، بدل تعشيش الكود كله جوه شرط if كبير.

٧. المكونات الأساسية للكود المقروء

- التنسيق (Formatting): مسافات، فواصل أسطر، طول السطر المعقول (عادة أقل من ٨٠-١٠٠ حرف).
- التنظيم (Organization): ترتيب منطقي للدوال والكلاسات جوه الملف، من العام للتفصيلي.
- التعليقات (Comments): توضيح الـ لماذا مش الـ ماذا، لأن الكود نفسه المفروض يوضح الـ ماذا.
- بنية التحكم البسيطة: تجنب التعشيش العميق واستخدام Early Returns وGuard Clauses.

٨. إزاي المبدأ ده بيشتغل داخليًا في عملية الفهم

الدماغ البشري بيقدر يحمل عدد محدود من العناصر في الذاكرة قصيرة المدى (Working Memory) في نفس الوقت. كل مستوى تعشيش إضافي، أو كل قفزة بين أجزاء كود متباعدة، بيستهلك من السعة المحدودة دي. الكود المنظم والمسطح (Flat) بيقلل الحمل على الذاكرة قصيرة المدى، وده اللي بيخلي القارئ يقدر يتابع المنطق كله من غير ما يفقد الخيط.

٩. إزاي بيتم استخدامه داخل الشركات الكبرى

شركات كبرى زي Google بتفرض أدوات تنسيق أوتوماتيكي (زي black أو gofmt حسب اللغة) كجزء إلزامي من الـ CI Pipeline، يعني الكود مش هيتقبل في الـ Pull Request أصلاً لو مخالف للتنسيق الموحد. ده بيشيل النقاش عن التنسيق تمامًا من عملية الـ Code Review ويخلي المراجعين يركزوا بس على المنطق والتصميم.

١٠. فين موقعه داخل مشروع ML حقيقي

موجود في كل ملف كود بيتكتب، لكنه حرج بشكل خاص في ملفات الـ Feature Engineering وTraining Loops، لأن دي غالبًا الملفات اللي بتتعدل بأكبر تكرار مع كل تجربة (Experiment) جديدة، فقابلية قراءتها بتحدد سرعة التجريب (Experimentation Velocity) للفريق كله.

١١. امتى الاهتمام بالـ Readability يكون أولوية قصوى

- أي كود هيدخل الـ Repository الرسمي أو هيتشارك مع فريق.
- أي كود متوقع له تعديلات متكررة مستقبلًا (زي Feature Engineering Pipelines).
- أي كود تعليمي أو مرجعي هيتعلم منه مهندسين جدد.

١٢. امتى ممكن تتساهل شوية

- في كود تجريبي سريع جدًا (Throwaway Script) هيتشغل مرة واحدة وينحذف بعدها.
- لكن لازم الحذر، لأن الكود التجريبي غالبًا بيعيش أطول بكتير من المتوقع.

١٣. أفضل الممارسات

Best Practices

- استخدم أداة تنسيق أوتوماتيكي زي black في Python عشان تضمن اتساق التنسيق من غير جدل شخصي.
- قلل مستويات التعشيش باستخدام Early Returns وGuard Clauses.
- خلي طول السطر معقول (عادة أقل من ٨٨-١٠٠ حرف حسب معيار الفريق).
- اكتب تعليقات توضح لماذا القرار اتاخد كده، مش ماذا الكود بيعمل (لأن ده المفروض يبان من الكود نفسه).

١٤. أشهر الأخطاء الشائعة

Common Mistakes

- تعشيش عميق جدًا من الشروط والحلقات (أكتر من ٣-٤ مستويات).
- أسطر طويلة جدًا بتحتاج Scroll أفقي عشان تتقرا.
- خلط أساليب تنسيق مختلفة في نفس الملف (مسافات وTabs مثلًا).
- تعليقات بتشرح حاجة واضحة أصلًا من الكود، بدل ما توضح سبب القرار.

١٥. اعتبارات الأداء

التنسيق والقابلية للقراءة مالهمش أي تأثير على أداء البرنامج وقت التشغيل، لأن دول بيتشالوا وقت التصريف أو التفسير (Compile/Interpretation) ومش ليهم وجود في الكود المنفذ فعليًا. الفايدة كلها في سرعة الفهم والصيانة، مش في سرعة التنفيذ.

١٦. الاعتبارات الأمنية

كود مقروء وواضح بيسهل اكتشاف الثغرات الأمنية المحتملة أثناء الـ Code Review، لأن المراجع بيقدر يتابع منطق التحقق من صحة المدخلات بسهولة. الكود المعقد والمتعشش بيخبي أحيانًا حالات حرجة (Edge Cases) ممكن تتسبب في ثغرات أمنية من غير ما حد يلاحظها.

١٧. اعتبارات قابلية التوسع

لما الفريق بيكبر، الكود المقروء بيخلي أي مهندس جديد يقدر يساهم بسرعة من غير حاجة لشرح مطول من زمايله. الاتساق في التنسيق عبر المشروع كله بيسهل كمان استخدام أدوات أوتوماتيكية (زي أدوات تحليل الكود الثابت Static Analysis) بشكل موحد.

١٨. مقارنة مع نهج بديل

البديل هو الاعتماد على تفضيلات شخصية غير موحدة لكل مهندس. الأسلوب ده ممكن يكون مقبول في مشروع فردي صغير، لكنه بيفشل تمامًا في فريق من أكتر من شخص، لأن كل انتقال بين كود مختلف الأسلوب بيستهلك وقت وجهد إضافي. لذلك الشركات الكبرى بتفرض معيار تنسيق واحد إلزامي بغض النظر عن التفضيل الشخصي لأي مهندس.

١٩. مثال عملي صغير

```
# Before - deep nesting, hard to follow
def get_discount(user):
    if user is not None:
        if user.is_active:
            if user.orders_count > 10:
                return 0.2
            else:
                return 0.1
        else:
            return 0
    else:
        return 0

# After - early returns, flattened logic
def get_discount(user):
    if user is None or not user.is_active:
        return 0
    if user.orders_count > 10:
        return 0.2
    return 0.1
```

٢٠. مثال من مشروع Production حقيقي

في مشاريع كبرى، ملف الـ Training Loop غالبًا بيتقسم بصريًا لأقسام واضحة بمسافات فاصلة: تجهيز البيانات، ثم حلقة التدريب نفسها، ثم حساب المقاييس، ثم الحفظ. كل قسم ليه تعليق عنوان بسيط، وده بيخلي أي مهندس يقدر يلاقي الجزء اللي محتاج يعدله في ثواني من غير ما يقرأ الملف كله.

٢١. ملخص الموضوع

خلاصة Readability

- الكود بيتقرأ أكتر بكتير مما بيتكتب، فالاستثمار في وضوحه بيرجع بفايدة أكبر بكتير.
- استخدام أداة تنسيق أوتوماتيكي بيشيل الجدل الشخصي حول الأسلوب.
- تقليل التعشيش واستخدام Early Returns بيقلل الحمل الذهني على القارئ.
- التعليقات لازم توضح لماذا، مش ماذا.

١,٥ Maintainability — قابلية الصيانة

Maintainability هي الغاية النهائية من كل المبادئ اللي اتكلمنا عنها لحد دلوقتي. Naming وFunctions وClasses وReadability كلهم أدوات، والهدف النهائي منهم إن الكود يفضل قابل للتعديل والتوسع بأمان على مدار سنين، مش بس شغال دلوقتي.

١. ليه قابلية الصيانة مهمة؟

أي مشروع Machine Learning حقيقي مش بيتكتب مرة واحدة وينتهي، هو بيتطور باستمرار: بيانات جديدة، نماذج جديدة، متطلبات عمل جديدة. لو الكود مش قابل للصيانة، كل تغيير بسيط بيتحول لعملية محفوفة بالمخاطر ومكلفة في الوقت، وده بيبطئ سرعة الفريق في تسليم قيمة جديدة (Feature Velocity) بشكل تراكمي مع الوقت.

المصطلح التقني اللي بيوصف تراكم التكاليف دي هو Technical Debt: كل قرار سريع أو غير منظم بياخد وقته دلوقتي بس بيرجع بفوايد سلبية (دين) لازم يتسدد لاحقًا بجهد أكبر.

٢. المشكلة اللي بتحلها قابلية الصيانة العالية

المشكلة هي الأنظمة اللي بتوصل لمرحلة يخاف فيها أي مهندس يعدل فيها، لأن أي تغيير بسيط ممكن يكسر حاجة تانية من غير ما يكون واضح ليه. الحالة دي بتتوصف أحيانًا بـ Legacy Code Fear، وبتؤدي لتباطؤ شديد في تطوير المنتج، وأحيانًا لقرار مكلف بإعادة كتابة النظام بالكامل من الصفر.

٣. إزاي كانت الشركات بتتعامل قبل الاهتمام بقابلية الصيانة

في مراحل النمو السريع، كتير من الشركات الناشئة كانت بتعطي أولوية قصوى للسرعة (Move Fast) على حساب جودة الكود، بحجة إن السرعة دلوقتي أهم من الصيانة لاحقًا. الاستراتيجية دي بتنفع في مرحلة إثبات الفكرة (Proof of Concept) لكنها بتصبح خطيرة جدًا لما المنتج ينجح ويكبر.

٤. ليه الطريقة القديمة مش مناسبة على المدى الطويل

تجاهل قابلية الصيانة بيخلق دين تقني بيتراكم بمعدل مركب: كل ميزة جديدة بتتبني فوق أساس هش، وكل تعديل بيزود الهشاشة دي أكتر. في النهاية بيوصل الفريق لنقطة معظم وقته بيروح في إصلاح مشاكل قديمة بدل بناء ميزات جديدة، وده اللي بيخلي شركات كتير تضطر لمشاريع إعادة هيكلة (Refactoring) ضخمة ومكلفة.

٥. الفكرة الأساسية وراء قابلية الصيانة

قابلية الصيانة العالية معناها إن أي مهندس (حتى لو مش هو اللي كتب الكود الأصلي) يقدر يفهم، يعدل، ويوسع الكود بثقة وبأقل قدر من المخاطرة. ده بيتحقق من تراكم كل المبادئ اللي اتكلمنا عنها: أسماء واضحة، دوال صغيرة، كلاسات متماسكة، وكود مقروء - بالإضافة لعنصر جديد هنا: التوثيق واختبارات تحمي من كسر السلوك المتوقع (وهو موضوع هنتعمق فيه في فصل الـ Testing لاحقًا).

٦. المفاهيم الأساسية

- Technical Debt: التكلفة المستقبلية الناتجة عن قرارات سريعة أو غير منظمة اتخدت في الماضي.
- Refactoring: عملية تحسين البنية الداخلية للكود من غير تغيير سلوكه الخارجي، بهدف تحسين قابلية الصيانة.
- Code Smells: علامات تحذيرية في الكود (زي دالة طويلة جدًا أو تكرار منطق) بتدل على مشاكل محتملة في قابلية الصيانة قبل ما تتحول لمشكلة حقيقية.
- Backward Compatibility: القدرة على تغيير أو تحسين جزء من النظام من غير كسر الأجزاء التانية اللي بتعتمد عليه.

٧. المكونات الأساسية لكود قابل للصيانة

- بنية واضحة ومنظمة (نتيجة تطبيق Naming وFunctions وClasses وReadability بشكل متسق).
- توثيق كافٍ (Documentation) يشرح القرارات المعمارية المهمة والغرض من كل وحدة.

- اختبارات (Tests) تحمي من كسر السلوك المتوقع عند أي تعديل مستقبلي.
- إدارة واضحة للتبعيات (Dependencies) والإصدارات، عشان تقدر تحدث مكتبة من غير ما تكسر النظام كله.

٨. إزاي المبدأ ده بيشتغل داخليًا

قابلية الصيانة مش خاصية منفصلة تضاف للكود، هي نتيجة تراكمية لكل قرار صغير في التصميم. كل اسم واضح، كل دالة صغيرة، كل كلاس متماسك، بيقلل تكلفة الفهم والتعديل شوية. القيمة الحقيقية بتظهر بمرور الوقت: مشروع مكتوب صح بيفضل سهل التعديل حتى بعد سنين، بينما مشروع مكتوب بسرعة بيبقى أصعب وأصعب في التعديل مع كل إضافة جديدة.

٩. إزاي بيتم استخدامه داخل الشركات الكبرى

شركات زي Google وAmazon بتخصص وقت مخصص (غالبًا بيتسمى Tech Debt Sprints أو Maintenance Time) في خطة الفريق بشكل دوري عشان يتعاملوا مع الـ Technical Debt المتراكم بدل ما يسيبوه يتراكم لحد ما يبقى غير قابل للسيطرة عليه. كمان بيتم قياس مؤشرات زي Code Coverage ومدى تعقيد الكود (Cyclomatic Complexity) بشكل دوري كجزء من مراقبة صحة الـ Codebase.

١٠. فين موقعه داخل مشروع ML حقيقي

قابلية الصيانة حرجة بشكل خاص في الأنظمة اللي بتشتغل لفترة طويلة في Production زي أنظمة الـ Fraud Detection أو الـ Recommendation Systems، لأن دي أنظمة بتحتاج تحديث مستمر مع تغير سلوك المستخدمين وظهور أنماط احتيال جديدة، وأي صعوبة في التعديل بتبطئ استجابة الفريق للتهديدات أو الفرص الجديدة.

١١. امتى الاستثمار في قابلية الصيانة يكون أولوية

- أي نظام متوقع يشتغل ويتطور لفترة طويلة (شهور أو سنين).
- أي نظام هيشتغل عليه أكتر من مهندس أو أكتر من فريق.
- أي نظام حرج للعمل (Business-Critical) بيتطلب تحديثات متكررة وآمنة.

١٢. امتى ممكن تقبل مستوى أقل من الاهتمام بالصيانة

- في تجربة سريعة (Proof of Concept) هدفها إثبات فكرة قبل الاستثمار الكامل فيها.
- لكن لازم يكون فيه خطة واضحة: لو الفكرة نجحت، الكود المؤقت لازم يتعاد كتابته بمعايير احترافية قبل ما يدخل Production.

١٣. أفضل الممارسات

Best Practices

- خصص وقت دوري ثابت للتعامل مع الـ Technical Debt بدل تركه يتراكم.
- اكتب اختبارات تحمي من كسر السلوك عند أي تعديل مستقبلي (هيتوسع الموضوع في فصل الـ Testing).
- وثق القرارات المعمارية المهمة (Architecture Decision Records) عشان أي مهندس جديد يفهم ليه اتخد قرار معين.
- راقب مؤشرات صحة الكود بشكل دوري زي Code Coverage وتعقيد الدوال.

١٤. أشهر الأخطاء الشائعة

Common Mistakes

- تجاهل الـ Technical Debt تمامًا بحجة السرعة، لحد ما يوصل لمرحلة صعبة جدًا في التعامل.
- إعادة كتابة النظام بالكامل من الصفر (Full Rewrite) كحل أول بدل التحسين التدريجي (Incremental Refactoring).

- عدم وجود اختبارات، وده بيخلي أي تعديل مستقبلي محفوف بالمخاطر لأن مفيش شبكة أمان تتأكد إن السلوك القديم لسه شغال.
- توثيق قديم غير محدث بيضلل المهندسين الجدد بدل ما يساعدهم.

١٥. اعتبارات الأداء

قابلية الصيانة والأداء مش دايمًا في نفس الاتجاه: أحيانًا التحسين الشديد للأداء (زي كتابة كود منخفض المستوى شديد التعقيد) بيقلل قابلية الصيانة. القرار الصح هو تحسين الأداء بس في الأجزاء اللي فعلاً حرجة (بناءً على قياس فعلي Profiling)، وترك باقي الكود بسيط وقابل للصيانة.

١٦. الاعتبارات الأمنية

الكود القابل للصيانة بيسهل تطبيق تحديثات أمنية بسرعة لما تظهر ثغرة جديدة أو تحديث مطلوب في مكتبة معينة. النظام المعقد وصعب الصيانة بياخد وقت أطول بكتير في تطبيق تحديثات أمنية عاجلة، وده بيزود فترة التعرض للخطر (Exposure Window).

١٧. اعتبارات قابلية التوسع

قابلية الصيانة العالية شرط أساسي لأي توسع مستقبلي في المشروع، سواء توسع في حجم البيانات، عدد المستخدمين، أو حتى حجم الفريق نفسه. نظام صعب الصيانة بيقاوم أي محاولة توسع لأن التعديلات البسيطة نفسها بتاخد وقت طويل ومخاطرة عالية.

١٨. مقارنة مع نهج بديل

البديل هو إعطاء الأولوية القصوى للسرعة الفورية على حساب الجودة طول الوقت (وليس فقط في مرحلة الـ Proof of Concept). الأسلوب ده بيدي نتائج سريعة في المدى القصير جدًا، لكنه بيتحول لعبء متراكم بيبطئ الفريق بشكل متزايد مع الوقت. الشركات الكبرى الناجحة بتوازن بين السرعة والجودة، وبتعتبر الاستثمار في قابلية الصيانة استثمار طويل المدى في سرعة الفريق نفسها.

١٩. مثال عملي صغير

```
# Before - duplicated validation logic across functions
def process_order_v1(order):
    if order['amount'] <= 0:
        raise ValueError('Invalid amount')
    # ... باقي المنطق

def process_order_v2(order):
    if order['amount'] <= 0:
        raise ValueError('Invalid amount')
    # ... منطق مختلف بس نفس التحقق مكرر

# After - validation extracted into a shared reusable function
def validate_order(order):
    if order['amount'] <= 0:
        raise ValueError('Invalid amount')

def process_order_v1(order):
    validate_order(order)
    # ... باقي المنطق

def process_order_v2(order):
    validate_order(order)
    # ... منطق مختلف بس التحقق موحد الآن
```

٢٠. مثال من مشروع Production حقيقي

في مشروع ML طويل الأمد، فريق بيحدث نموذج الـ Fraud Detection كل أسبوعين ببيانات جديدة. لو الـ Pipeline مكتوب بمعايير قابلية الصيانة العالية (دوال صغيرة، اختبارات تحمي كل مرحلة، توثيق واضح)، التحديث الأسبوعي ده بياخد ساعات. لو مكتوب بشكل سيء، نفس التحديث ممكن ياخد أيام كاملة من التتبع والتصحيح كل مرة.

٢١. ملخص الموضوع

خلاصة Maintainability

- قابلية الصيانة هي النتيجة التراكمية لكل مبادئ Clean Code مجتمعة مع بعض.
- الدين التقني (Technical Debt) بيتراكم بمعدل مركب لو اتجاهل، فلازم يتعامل معاه بشكل دوري.
- الاختبارات والتوثيق شبكة أمان أساسية لأي تعديل مستقبلي آمن.
- التوازن بين السرعة والجودة استراتيجية طويلة المدى، مش اختيار لمرة واحدة.

مشروع عملي على الفصل: إعادة هيكلة Data Preprocessing Script

المشروع ده هدفه إنك تطبق كل مبادئ الفصل (Naming، Functions، Classes، Readability، Maintainability) على سكربت واحد بسيط لمعالجة بيانات، بحيث تشوف الفرق العملي بين كود شغال بس وكود احترافي.

السيناريو: عندك سكربت واحد بيقرأ ملف CSV لبيانات عملاء، بيشيل القيم الناقصة، بيطبع القيم الرقمية (Normalization)، وبيحفظ الناتج في ملف جديد. السكربت الأصلي مكتوب بسرعة وفيه كل مشاكل الفصل: أسماء غامضة، دالة واحدة عملاقة، تعشيش عميق، ومفيهوش أي تنظيم كائني.

خطوات التنفيذ

- الخطوة الأولى: افصل السكربت لدوال صغيرة، كل دالة بمسؤولية واحدة (تحميل، تنظيف، تطبيع، حفظ).
- الخطوة الثانية: أعد تسمية كل المتغيرات والدوال بأسماء توضح النية بوضوح (طبق مبادئ Naming).
- الخطوة الثالثة: جمّع العملية كلها في كلاس واحد اسمه CustomerDataPreprocessor، بحيث الحالة (مسار الملف، البيانات المحملة) محفوظة كـ Attributes والعمليات كـ Methods.
- الخطوة الرابعة: نظم التنسيق وقلل التعشيش باستخدام Early Returns وGuard Clauses، وشغل أداة تنسيق أوتوماتيكي زي black.
- الخطوة الخامسة: اكتب Docstring لكل Method عام، ووثق أي قرار غير بديهي (زي طريقة معينة في التعامل مع القيم الناقصة).

تحديات شائعة عند التطبيق

- المبالغة في التقسيم (Over-Engineering): بعض المبتدئين بيقسموا الكود لدرجة مفرطة (دالة لكل سطر تقريبًا)، وده بيزود التعقيد بدل ما يقلله. القاعدة: قسم لما التقسيم يزود الوضوح فعلًا، مش بشكل آلي.
- الخوف من إعادة الهيكلة على كود Production شغال: كتير من المهندسين بيترددوا يعيدوا هيكلة كود قديم بيشتغل، خوفًا من كسره. الحل هو وجود اختبارات (Tests) تتأكد إن السلوك متغيرش بعد إعادة الهيكلة، وهو موضوع هنتعمق فيه في فصل الـ Testing.
- التوازن بين الوقت المتاح وجودة الكود: في بيئة عمل حقيقية، مش دايمًا هيكون عندك وقت كافٍ لتطبيق كل مبدأ بشكل مثالي. المهم إنك تحدد أولويات الاهتمام حسب أهمية الكود (كود هيتشارك أو هيشتغل طويل الأمد، أهم من سكربت تجريبي مؤقت).

إزاي الفصل ده بيتستخدم داخل مشروع Machine Learning حقيقي

كل مبدأ في الفصل ده بيظهر يوميًا في شغل ML Engineer حقيقي: من أول لحظة تكتب فيها دالة لتحميل بيانات، أو كلاس يغلف نموذج، إنت بتطبق Naming وFunctions وClasses. ولما فريقك بيراجع الـ Pull Request بتاعك، Readability هي أول حاجة بتحدد سرعة الموافقة على الكود. ولما المشروع يكبر ويحتاج تحديثات مستمرة، Maintainability هي اللي بتحدد سرعة استجابة الفريق لأي تغيير مطلوب.

في الشركات الكبرى، مفيش مشروع ML بيوصل لمرحلة Production من غير إن الكود يمر بمراجعة صارمة تتأكد من الالتزام بكل المبادئ دي، لأن الكود ده هيتشغل ويتصان لفترة طويلة من فريق كامل، مش هيفضل ملك مهندس واحد.

ملخص الفصل الأول وربطه بالفصل القادم

في الفصل ده اتعلمت إزاي تحول كود بيشتغل بس لكود احترافي قابل للفهم والصيانة والتوسع، من خلال خمس ركائز: Naming (أسماء واضحة تحمل المعنى)، Functions (دوال صغيرة بمسؤولية واحدة)، Classes (تجميع منطقي للحالة والسلوك)، Readability (تنظيم يقلل الحمل الذهني على القارئ)، وMaintainability (النتيجة التراكمية لكل ده على المدى الطويل).

الفصل الجاي هيبني مباشرة على أساس الكلاسات اللي اتعلمناه هنا. SOLID Principles هي في الآخر خمس قواعد بتحكم إزاي تصمم كلاسات وعلاقاتها ببعض بشكل يخليها مرنة وقابلة للتوسع من غير ما تحتاج تعدل كود موجود بالفعل. يعني كل حاجة اتعلمتها في Classes هنا (Composition، Coupling، Cohesion) هتتحول لمبادئ رسمية ودقيقة في الفصل الجاي، وهتستخدمها بعد كده في Design Patterns اللي هي في الأساس تطبيقات عملية لمبادئ SOLID دي على مشاكل تصميم متكررة.